ROADMAP

AI 업무자동화·서비스·에이전트 개발 로드맵

이 스터디의 기준 문서입니다. 로드맵은 계속 고쳐 가는 문서라 버전으로 관리하고, 사이트도 같은 번호를 씁니다.

문서 버전 v1.2 · 2026-11-12 · 24세션 기준, 모든 실습을 Python · Java 트랙으로 제공 · 스터디 사이트(ai-study)는 이 버전을 따릅니다. 변경 내역은 맨 아래 '버전 기록'에 있습니다.

개요

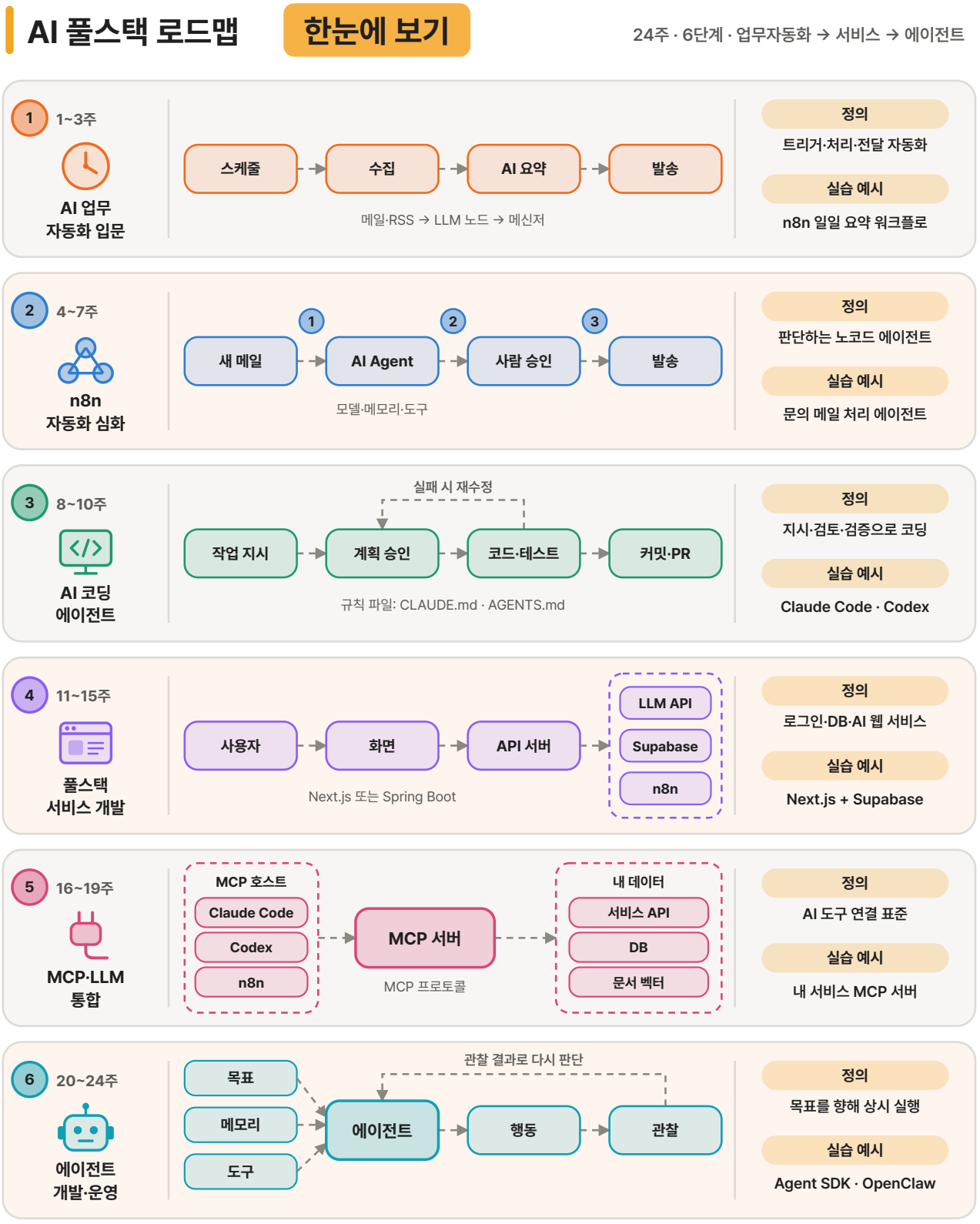
24주, 6단계 과정으로 "노코드 업무자동화 → AI 코딩으로 서비스 개발 → 직접 만든 에이전트 운영"까지 한 사람이 끝까지 해내는 것이 목표입니다. 단계마다 실제로 돌아가는 결과물을 하나씩 남깁니다.

- **흐름**: 쉬운 것(n8n 자동화)에서 시작해 코드(Claude Code·Codex)로 넘어가고, 연결 표준(MCP)으로 묶은 뒤, 상시 동작 에이전트(Agent SDK·OpenClaw)로 마무리합니다.
- **대상**: 기본적인 IT 지식은 있지만 웹 서비스·에이전트 개발 경험은 적은 실무자·리더.
- **시간**: 주 5~8시간(학습 40% + 실습 60%).
- **준비물**: GitHub 계정, Claude 또는 ChatGPT 유료 플랜(코딩 에이전트 사용량), Docker 설치된 PC, 연습용 클라우드 계정(무료 티어).

두 가지 언어 트랙: 4·5단계는 둘 중 하나를 고릅니다. 1~3단계와 6단계는 언어와 무관하게 같습니다.

트랙	언어·프레임워크	맞는 경우
기본 트랙	TypeScript(Next.js) + Python	빠른 PoC, 혼자 끝까지 만들어 보기, 클라우드 무료 배포
Java/Spring 트랙	Java 21 + Spring Boot + Spring AI	운영 시스템이 Spring, Oracle·사내 인증·폐쇄망 배포를 그대로 써야 할 때

한눈에 보기



↔ 옆으로 밀어 보기 · 로드맵 · 6단계, 24주

1~2단계는 노코드 자동화, 3~4단계는 AI 코딩으로 서비스 개발, 5~6단계는 MCP로 연결한 에이전트 개발입니다.

도구 지도

모델사별 도구가 같은 층으로 정렬되고, MCP가 모두를 잇습니다

열 = 회사 · 행 = 도구 층 · 2026년 10월 기준

도구 층	Anthropic	OpenAI	Google	오픈소스·기타
대화·업무 앱	**Claude 앱** 프로젝트·커넥터·스킬	**ChatGPT** GPTs·커넥터	**Gemini 앱** Workspace 연동	**Ollama 등** 로컬 모델 실행
터미널 코딩 에이전트 (CLI)	**Claude Code** 서브에이전트·훅·스킬	**Codex CLI** 오픈소스·샌드박스	**Antigravity CLI** 구 Gemini CLI · agy	**OpenCode** Copilot CLI · 모델 선택형
프로젝트 규칙 파일	**CLAUDE.md**	**AGENTS.md**	**GEMINI.md 호환**	**AGENTS.md 공용**
에이전트 SDK · API	**Claude Agent SDK** Messages API	**Agents SDK** Responses·Agents API	**Gemini API** Antigravity 2.0	**OpenClaw** 상시 개인 에이전트
Java·Spring 연동	**Spring AI 2.0 · MCP Java SDK 1.0 · LangChain4j** Spring Boot에서 Claude·GPT·Gemini·오픈 모델을 같은 ChatClient로 호출, @McpTool로 MCP 서버 구현			
업무자동화	**n8n · 노코드 워크플로** 위 모든 회사의 모델·API·MCP를 노드로 조합 (AI Agent 노드, MCP Client·Server)			
연결 표준	**MCP · 도구 연결 표준** Claude Code · Codex CLI · Antigravity CLI · n8n · Spring AI 모두 같은 MCP 서버를 연결			
LLM 모델	**Claude** Opus · Sonnet · Haiku	**GPT** GPT-5.5 등	**Gemini** Gemini 3.5 등	**오픈 모델** 로컬·사내 구축

· Gemini CLI는 2026년 6월 18일 개인 사용자 지원 종료 → Antigravity CLI(agy)로 이전 (유료 API 키로는 계속 사용 가능)

· OpenAI AgentKit은 2026년 11월 30일 종료 예정 → 코드형은 Agents SDK, OpenAI가 루프를 돌리는 Agents API는 공개 베타

↔ **옆으로 밀어 보기** · 도구 지도 · 4개 층 + 모델·개발 도구

에이전트는 모델에게 판단을 맡기고 MCP를 거쳐 도구·데이터를 씁니다. Claude Code·Codex는 이 층들을 만드는 도구이면서, 스스로도 MCP를 쓰는 에이전트입니다.

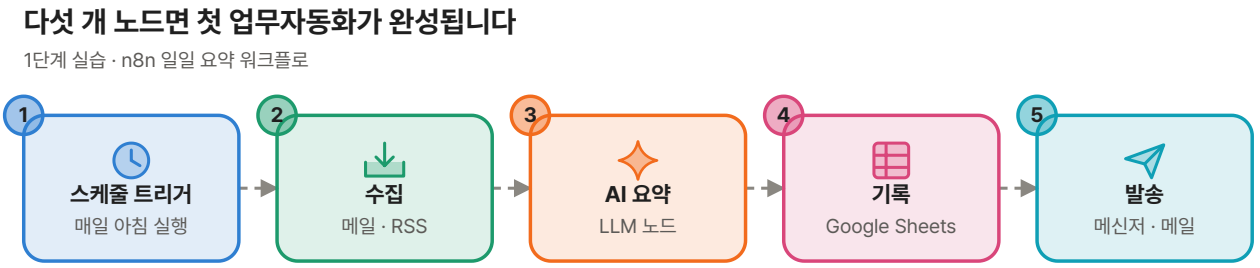
고르는 기준: 같은 행의 도구는 서로 바꿔 쓸 수 있습니다. 실습은 한 회사 열을 기본으로 정하고, 3단계에서 다른 열의 CLI 하나를 교차 리뷰용으로 함께 씁니다.

- **터미널 CLI**: Claude Code·Codex CLI·Antigravity CLI 모두 프로젝트 규칙 파일, 서브에이전트, MCP 연결을 지원합니다. Codex CLI와 OpenCode는 오픈소스입니다.
- **Google 전환**: Gemini CLI는 개인 사용자 대상 서비스를 2026년 6월 18일 종료하고 Antigravity CLI(`agy`)로 옮겼습니다. 기존 MCP 설정은 `agy plugin import gemini`로 가져올 수 있고 GEMINI.md도 계속 읽습니다. (Google 개발자 블로그, Flutter 문서)

- **OpenAI 에이전트 스택**: 새 프로젝트는 Responses API + Agents SDK 조합이 기본입니다. 루프를 OpenAI가 대신 돌리는 Agents API는 2026년 9월부터 공개 베타이고, AgentKit은 2026년 11월 30일 종료 예정입니다. (Apidog 비교, Classmethod 정리)

1단계 · AI 업무 활용과 자동화 입문 (1~3주)

반복 업무 3개를 골라 AI와 n8n으로 사람 손을 빼는 것이 목표입니다.



↔ 옆으로 밀어 보기 · 1단계 흐름 · 일일 요약 워크플로

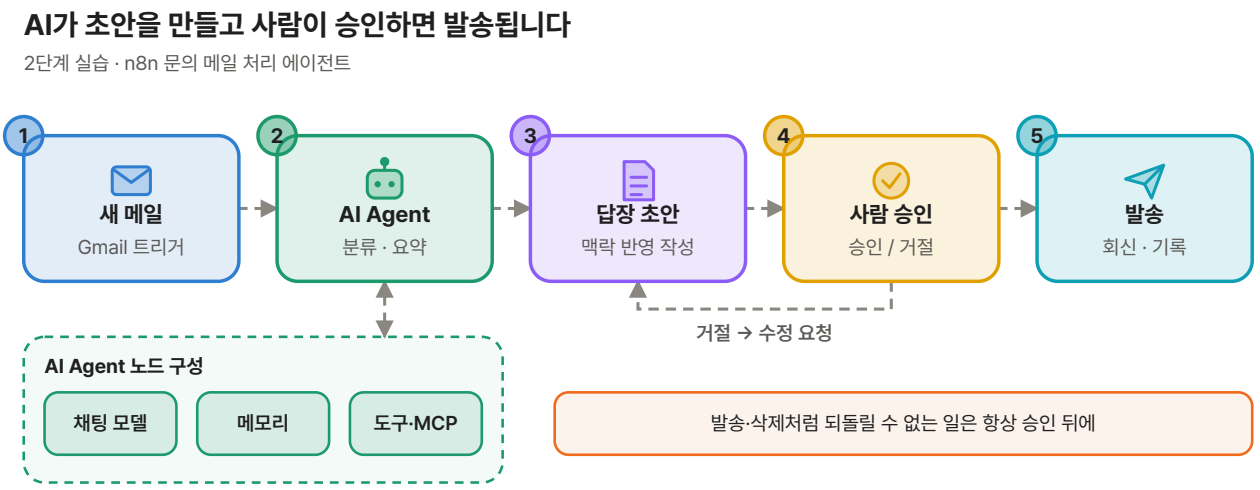
모든 자동화는 트리거 → 처리 → 전달의 같은 뼈대를 가집니다. 이 다섯 노드를 손에 익히면 나머지는 처리 단계를 바꾸는 일입니다.

- **AI 활용 기본기**: 프롬프트 4요소(역할·맥락·제약·형식), 예시 주기, 긴 문서 요약·비교, 출처 확인 습관.
- **자동화 사고법**: 업무를 "트리거 → 처리 → 결과 전달"로 쪼개기, 자동화 후보 고르기(빈도 × 소요시간).
- **n8n 시작**: Docker로 셀프호스팅 또는 클라우드 체험판, 노드·연결·실행 기록 이해, 스케줄·웹훅 트리거.
- **첫 워크플로**: 메일/RSS 수집 → LLM 요약 → 메신저·메일 발송, Google Sheets 기록.
- **AI 데스크톱 도구**: Claude·ChatGPT의 프로젝트, 커넥터, 스킬 기능으로 반복 작업 템플릿화.

산출물: 매일 아침 뉴스·메일 요약을 보내 주는 n8n 워크플로.

2단계 · n8n 워크플로 자동화 심화 (4~7주)

n8n 안에서 "판단하는 자동화", 즉 노코드 에이전트를 만드는 것이 목표입니다.



↔ 옆으로 밀어 보기 · 2단계 흐름 · 문의 메일 에이전트

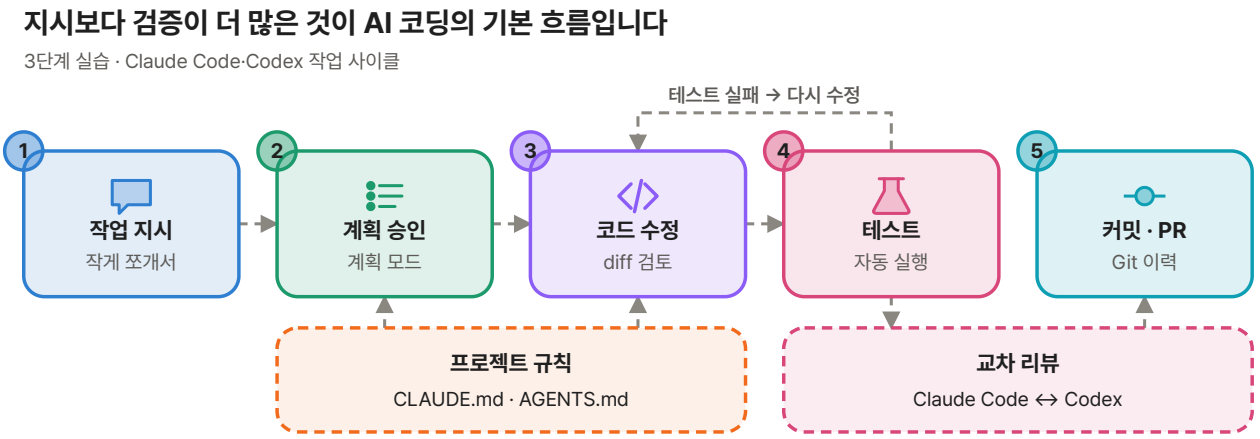
AI Agent 노드는 모델·메모리·도구를 조합해 판단하고, 거절된 초안은 수정 요청과 함께 다시 작성 단계로 돌아갑니다.

- **API·데이터 기초**: HTTP Request 노드, REST·JSON·인증(API Key, OAuth), 표현식으로 데이터 변환, Code 노드(JavaScript/Python).
- **AI Agent 노드**: 채팅 모델 + 메모리 + 도구 조합. n8n의 다른 노드(HTTP, DB 조회 등)를 에이전트 도구로 붙일 수 있습니다.
- **RAG 맛보기**: 문서를 벡터 저장소에 넣고 질문에 근거로 답하는 워크플로.
- **MCP 첫 경험**: MCP Client Tool 노드로 외부 MCP 서버의 도구를 에이전트에 연결, MCP Server Trigger로 내 워크플로를 MCP 도구로 공개.
- **운영 기본**: 오류 워크플로, 재시도, 서브워크플로, 실행 로그·자격증명 관리.

산출물: 문의 메일을 분류·요약하고 답장 초안을 만들어 승인 후 발송하는 n8n 에이전트.

3단계 · AI 코딩 에이전트: Claude Code·Codex (8~10주)

노코드로 막히는 지점을 코드로 넘는 단계입니다. 직접 타이핑보다 "지시·검토·검증" 능력을 기릅니다.



↔ 옆으로 밀어 보기 · 3단계 흐름 · 코딩 에이전트 사이클

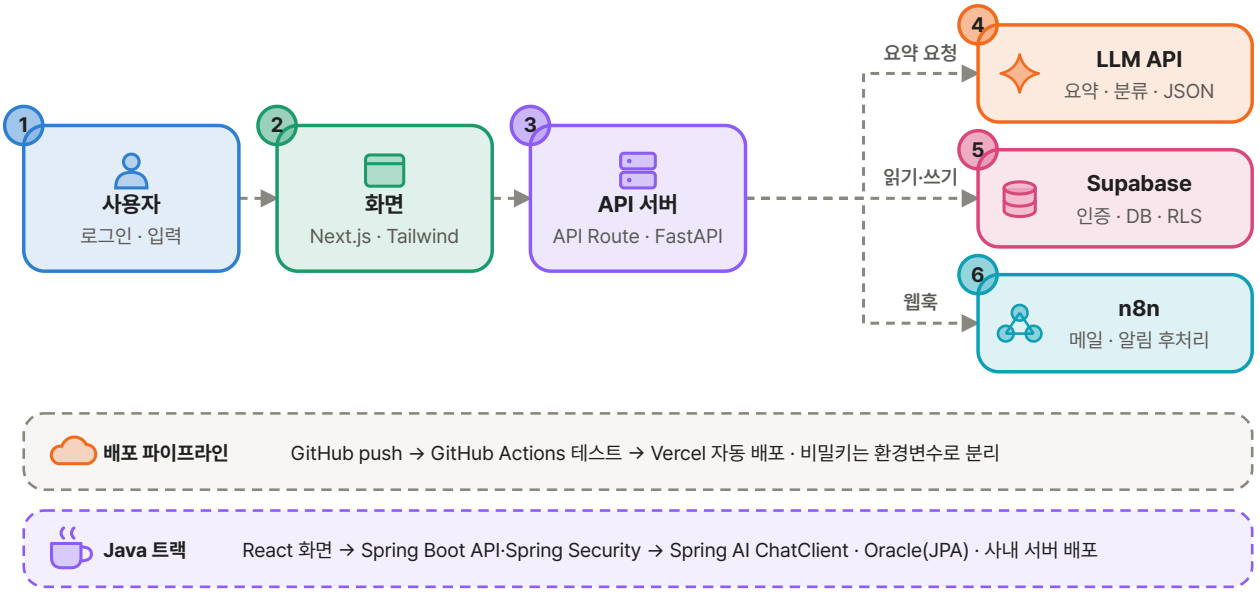
규칙 파일이 계획과 수정을 이끌고, 테스트를 통과한 변경만 다른 에이전트의 교차 리뷰를 거쳐 커밋됩니다.

- **개발 기초 최소치**: 터미널·Git(커밋, 브랜치, PR), Node.js·Python 실행 환경, 패키지 관리.
- **Claude Code**: 계획 모드로 설계 먼저 받기, CLAUDE.md로 프로젝트 규칙 고정, 슬래시 커맨드·스킬·훅, 서브에이전트, MCP 서버 연결.
- **Codex**: 터미널 CLI·IDE 확장·클라우드 에이전트·GitHub 연동의 여러 화면, AGENTS.md로 규칙 지정, 샌드박스·승인 모드 차이.
- **두 도구 함께 쓰기**: 한쪽이 작성, 다른 쪽이 리뷰. 같은 작업을 둘에 맡겨 결과 비교.
- **검증 습관**: 작은 단위 지시 → diff 검토 → 테스트 실행 → 커밋. AI가 만든 코드는 반드시 테스트로 확인.

산출물: 2단계 n8n 워크플로 일부를 Python 스크립트·CLI 도구로 다시 만든 저장소(테스트 포함).

4단계 · 풀스택 서비스 개발 (11~15주)

코딩 에이전트와 함께 로그인·DB·AI 기능이 있는 웹 서비스를 처음부터 배포까지 만드는 것이 목표입니다.



↔ 옆으로 밀어 보기 · 4단계 흐름 · 웹 서비스 요청 경로

API 서버가 중심입니다. 화면은 API만 부르고, API가 LLM·DB·n8n에 일을 나눠 줍니다.

- **프론트엔드**: Next.js(React) + Tailwind, 컴포넌트·상태·라우팅. 화면 시안은 AI로 먼저 뽑고 다듬기.
- **백엔드**: Next.js API Route 또는 FastAPI, REST 설계, 입력 검증, 오류 처리.
- **데이터**: Supabase(Postgres) — 테이블 설계, 인증, 행 단위 보안(RLS), 스토리지.
- **LLM 기능 넣기**: 모델 API 호출, 스트리밍 응답, JSON 구조화 출력, 토큰·비용 계산.
- **배포**: Vercel 등 호스팅, 환경변수·비밀키 분리, GitHub Actions로 자동 배포.
- **n8n 연동**: 서비스에서 웹훅으로 n8n 워크플로를 호출해 메일·알림 같은 후처리를 맡기기.

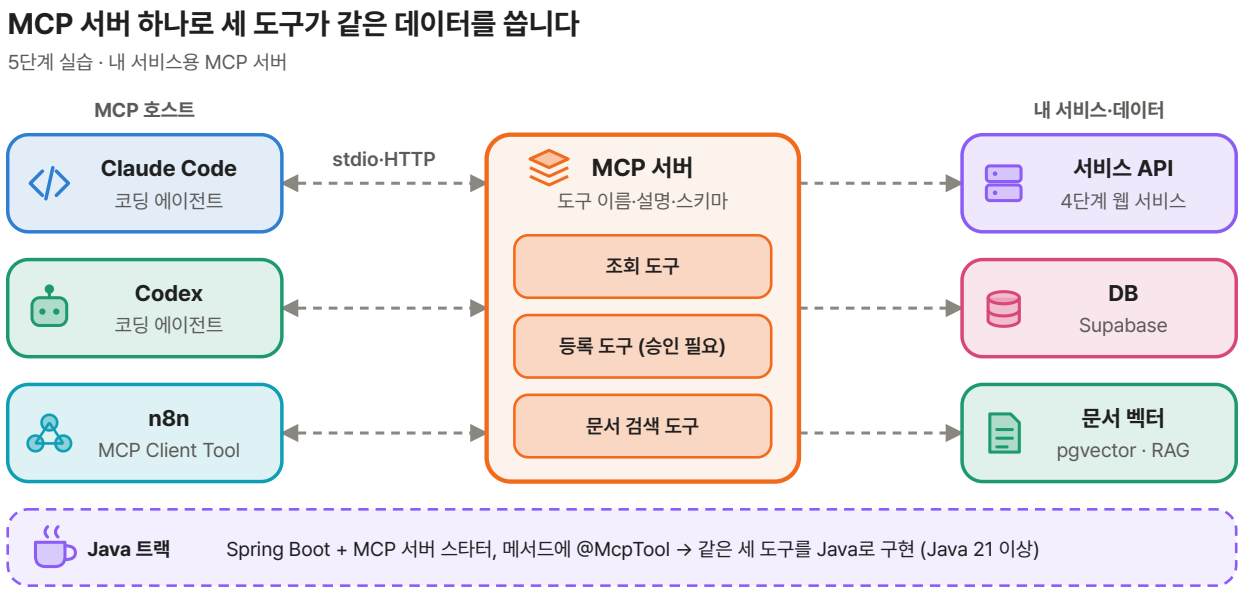
Java/Spring 트랙(선택): 같은 서비스를 기업 표준 스택으로 만듭니다.

- **백엔드**: Spring Boot REST API, 입력 검증, Spring Security로 로그인·권한.
- **데이터**: Oracle을 JDBC/JPA로 연결합니다. 실습은 Oracle Free 컨테이너나 Postgres로 대신할 수 있습니다.
- **LLM 기능**: Spring AI의 ChatClient로 모델을 호출하고, 구조화 출력은 Java 레코드로 받습니다. Claude·OpenAI·Gemini 모두 같은 인터페이스로 바꿔 끼울 수 있습니다.
- **화면**: React(또는 Next.js)로 프론트를 따로 두고 Spring API를 호출합니다.
- **배포**: Docker 이미지 → 사내 서버 또는 쿠버네티스. GitHub Actions 대신 사내 CI를 써도 흐름은 같습니다.
- **코딩 에이전트 활용**: Claude Code·Codex에 CLAUDE.md/AGENTS.md로 패키지 구조, 코딩 규칙, 테스트 명령(`./gradlew test`)을 알려 주면 Java 프로젝트도 똑같이 맡길 수 있습니다.

산출물: 로그인 → 데이터 입력 → AI 요약·분류 → 결과 저장·알림까지 되는 웹 서비스 v1.

5단계 · MCP 서버와 LLM 기능 통합 (16~19주)

MCP는 "AI가 쓸 수 있는 도구를 한 번 만들어 여러 클라이언트에서 재사용"하게 해 주는 표준입니다. 이 단계에서 내 서비스와 데이터를 AI가 직접 다룰 수 있게 엽니다.



↔ 옆으로 밀어 보기 · 5단계 흐름 · MCP 연결 구조

도구를 한 번만 만들면 Claude Code·Codex·n8n이 같은 방식으로 호출합니다. 쓰기 도구는 승인을 거치게 설계합니다.

- **MCP 개념**: 호스트(Claude Code·Codex·Claude 앱·n8n) ↔ 클라이언트 ↔ 서버, 서버가 제공하는 도구·리소스·프롬프트, 로컬(stdio)과 원격(HTTP) 방식.
- **MCP 서버 만들기**: 공식 SDK(TypeScript 또는 Python)로 4단계 서비스의 조회·등록 API를 도구로 감싸기. 도구 이름·설명·입력 스키마를 잘 쓰는 법.
- **연결 테스트**: 같은 서버를 Claude Code, Codex, n8n MCP Client Tool에 각각 붙여 동작 비교.
- **n8n을 MCP로**: MCP Server Trigger로 n8n 워크플로를 도구로 공개, n8n MCP 서버로 코딩 에이전트가 워크플로를 만들고 검증하게 하기.
- **RAG 고도화**: 문서 분할·임베딩·pgvector 검색, 근거 표시, 테스트 질문 세트로 정답률 측정.
- **보안**: 읽기/쓰기 도구 분리, 인증 토큰, 위험 작업은 확인 요청.

Java/Spring 트랙(선택): MCP 공식 Java SDK는 1.0 정식 버전이 나왔고 Spring AI 팀과 함께 관리됩니다. Spring AI 2.0은 MCP 서버·클라이언트용 Spring Boot 스타터와 `@McpTool` 애너테이션을 제공해, 메서드에 애너테이션만 붙이면 도구 스키마가 자동으로 만들어집니다.

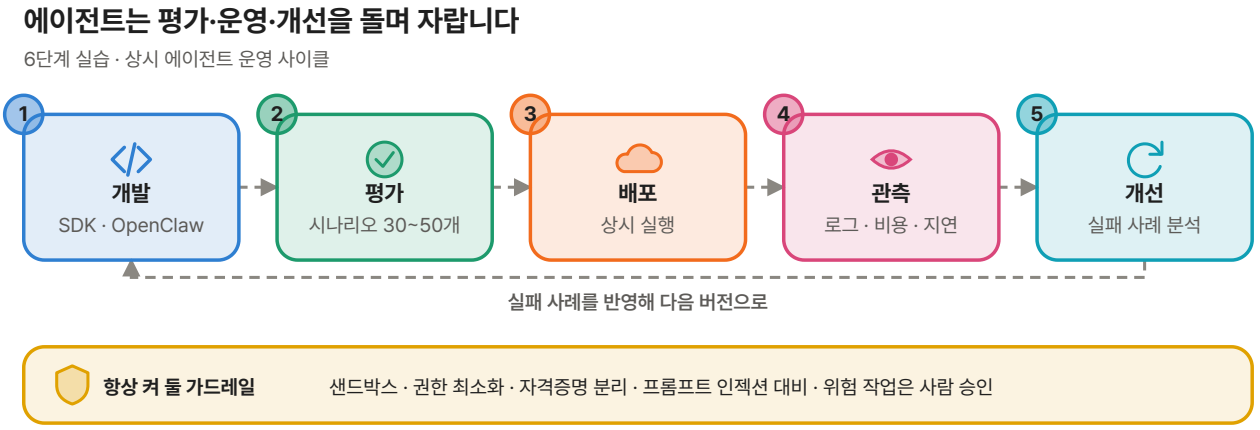
- **MCP 서버**: `spring-ai-starter-mcp-server-webmvc` + `@McpTool`로 Oracle 조회·등록 기능을 도구로 공개합니다. Java 21 이상이 필요합니다.

- **RAG**: Spring AI의 VectorStore(pgvector, Oracle 등)와 문서 리더로 같은 RAG를 구현합니다.
- **대안 라이브러리**: LangChain4j도 MCP와 여러 모델을 지원합니다.
- **출처**: MCP Java SDK 1.0 릴리스, Spring AI 2.0 MCP 애너테이션, Spring AI 2.0 무상태 MCP 서버

산출물: 내 서비스용 MCP 서버(도구 5개 이상) + 세 클라이언트에서 동작 확인 기록.

6단계 · 에이전트 개발·운영 (20~24주)

여러 단계 일을 스스로 계획·실행하고, 사람은 승인만 하는 에이전트를 직접 만들어 상시 운영하는 것이 목표입니다.



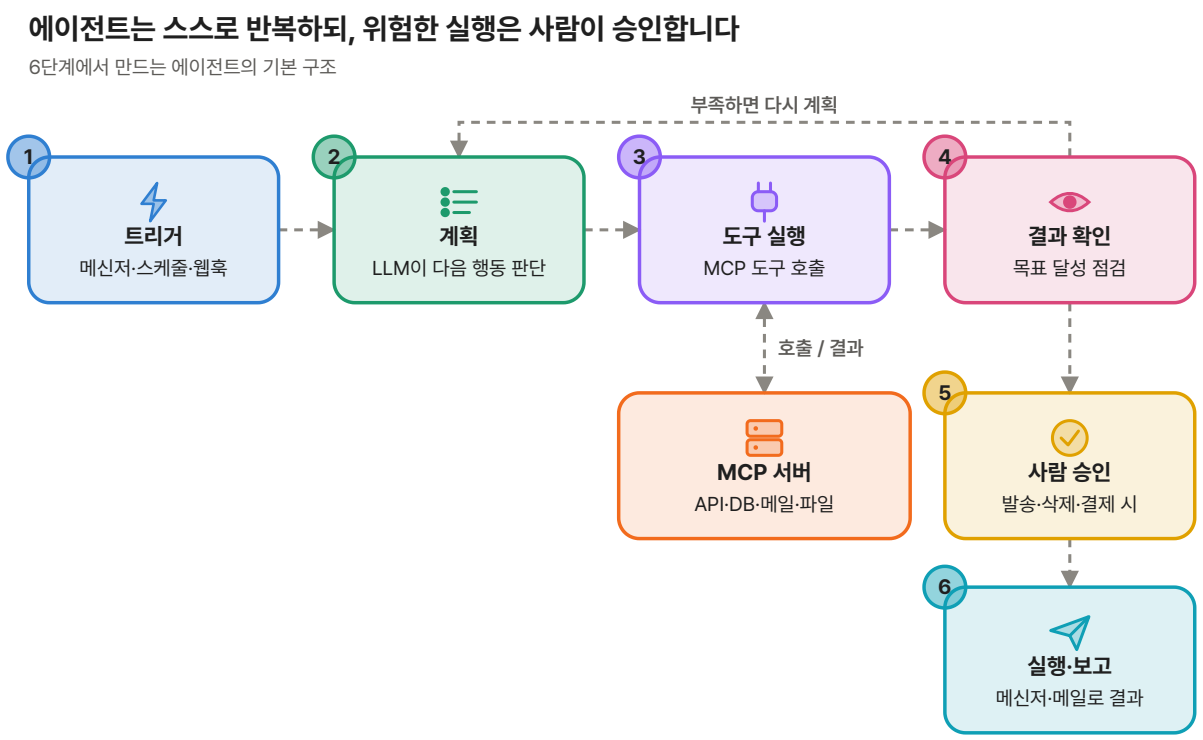
↔ 옆으로 밀어 보기 · 6단계 흐름 · 에이전트 운영 사이클

배포는 끝이 아니라 관측의 시작입니다. 로그에서 모은 실패 사례가 다음 평가 시나리오가 됩니다.

- **에이전트 원리**: 관찰 → 추론 → 도구 실행 → 결과 반영의 반복(에이전트 루프), 컨텍스트 관리, 장기 메모리.
- **코드로 만들기**: Claude Agent SDK 또는 OpenAI Agents SDK로 도구 호출 에이전트 작성, 5단계 MCP 서버를 도구로 연결, 서브에이전트로 역할 분리(조사·작성·검토).
- **OpenClaw로 상시 에이전트**: 내 PC·서버에서 계속 도는 오픈소스 개인 에이전트. 게이트웨이, 메신저 채널(Telegram·Slack 등), 스킬, 파일 기반 메모리 구조를 뜯어보며 실제 에이전트 아키텍처를 학습합니다. 모델은 Claude·GPT 등 선택형.
- **오케스트레이션 선택**: 정해진 흐름은 n8n, 판단이 많은 흐름은 코드 에이전트, 개인 비서형은 OpenClaw.
- **평가와 관측**: 테스트 시나리오 30~50개로 성공률 측정, 요청·도구 호출·비용 로그, 실패 사례 수집.
- **보안·가드레일**: 샌드박스·권한 최소화, 자격증명 분리, 프롬프트 인젝션 대비, 돈·삭제·발송은 반드시 사람 승인.

산출물: 메신저로 지시하면 내 서비스 데이터를 조회·정리해 보고서를 만들고, 승인 후 발송하는 상시 에이전트.

에이전트 동작 구조



↔ 옆으로 밀어 보기 · 에이전트 루프 · 4단계 반복 + 승인

계획→실행→확인을 목표가 달성될 때까지 반복하고, 되돌리기 어려운 작업 앞에는 반드시 승인 단계를 둡니다. n8n, Agent SDK, OpenClaw 모두 이 구조를 다른 방식으로 구현합니다.

핵심 도구 비교

다섯 도구는 경쟁 관계가 아니라 층이 다릅니다. n8n은 정해진 흐름, 코딩 에이전트는 만드는 일, MCP는 연결, OpenClaw는 상시 실행을 맡습니다.

도구	성격	주 용도	라이선스·비용	배우는 단계
n8n	워크플로 자동화 플랫폼	트리거 기반 업무자동화, AI Agent 노드, MCP 클라이언트·서버 양방향	소스 공개, 셀프호스팅 또는 클라우드 유료	1·2·5
Claude Code	Anthropic 코딩 에이전트	코드 작성·수정·테스트, 서브에이전트, MCP 연결	Claude 유료 플랜 또는 API	3~6
Codex	OpenAI 코딩 에이전트	CLI·IDE·클라우드·GitHub에서 같은 에이전트, MCP 지원	CLI는 오픈소스(Apache-2.0), ChatGPT 플랜 사용량 포함	3~6
MCP	AI–도구 연결 표준 프로토콜	내 서비스·데이터를 여러 AI 클라이언트에 도구로 공개	오픈 표준, 무료	2·5·6
OpenClaw	셀프호스팅 개인 에이전트 런타임	메신저로 지시받아 상시 실행, 스킬·메모리	MIT 오픈소스, 모델 사용료 별도	6

OpenClaw는 PC·서버에 대한 넓은 실행 권한을 가지므로 연습은 격리된 VM이나 별도 계정에서 하는 것을 권합니다.

마일스톤 체크리스트

항목을 모두 체크해야 다음 단계로 넘어갑니다. 결과물은 하나의 GitHub 조직(또는 저장소 묶음)에 모아 포트폴리오로 씁니다.

- [ ] 1단계: n8n 설치, 매일 도는 요약 워크플로 1개 이상
- [ ] 2단계: AI Agent 노드 + 도구 2개 이상 + 승인 단계가 있는 워크플로
- [ ] 3단계: Claude Code·Codex로 같은 기능 구현 후 비교 메모, 테스트 통과
- [ ] 4단계: 로그인·DB·AI 기능 있는 웹 서비스 배포 URL
- [ ] 5단계: 직접 만든 MCP 서버를 Claude Code·Codex·n8n에서 각각 호출 성공
- [ ] 6단계: 코드 에이전트 또는 OpenClaw 기반 상시 에이전트 운영 2주 이상
- [ ] 6단계: 테스트 시나리오 30개 이상 성공률 측정, 보안 체크리스트 점검

24세션 통합 커리큘럼 (v1.1)

16주 AI 기술 스터디 커리큘럼(기초·활용·심화·사업화)과 이 로드맵의 6단계를 하나로 합쳐 6파트 × 4세션으로 운영합니다. 세션마다 개념 · 이론(도식) · 실습 코드 · 실무 예제를 스터디 사이트에서 카드로 봅니다.

세션	파트	주제	출처
1	1. AI 기초	AI 개념 지도	커리큘럼 1주
2	1. AI 기초	LLM의 원리	커리큘럼 2주
3	1. AI 기초	프롬프트 엔지니어링	커리큘럼 3주 · 로드맵 1단계
4	1. AI 기초	생성형 AI 지형도와 도구 지도	커리큘럼 4주 · 도구 지도
5	2. 업무자동화	n8n 업무자동화 입문	로드맵 1단계
6	2. 업무자동화	LLM API와 구조화 출력	커리큘럼 5주
7	2. 업무자동화	n8n AI Agent 노드와 사람 승인	로드맵 2단계
8	2. 업무자동화	임베딩과 벡터DB	커리큘럼 6주
9	3. RAG·데이터	RAG 기초	커리큘럼 7주
10	3. RAG·데이터	고급 RAG와 평가	커리큘럼 12주
11	3. RAG·데이터	Text-to-SQL	커리큘럼 11주
12	3. RAG·데이터	데이터 품질과 메타데이터 정비	커리큘럼 11·12주 보강
13	4. AI 코딩·풀스택	AI 코딩 에이전트	커리큘럼 8주 · 로드맵 3단계
14	4. AI 코딩·풀스택	풀스택 ① 화면과 API (Next.js / Spring Boot)	로드맵 4단계
15	4. AI 코딩·풀스택	풀스택 ② DB·인증·AI 기능	로드맵 4단계
16	4. AI 코딩·풀스택	배포와 CI/CD	로드맵 4단계
17	5. 에이전트·MCP	에이전트 기초: 도구 호출과 루프	커리큘럼 9주
18	5. 에이전트·MCP	MCP 서버 만들기 (Python / Java)	커리큘럼 10주 · 로드맵 5단계
19	5. 에이전트·MCP	에이전트 개발: Agent SDK와 OpenClaw	로드맵 6단계
20	5. 에이전트·MCP	에이전트 운영·평가·가드레일	로드맵 6단계
21	6. 전문·사업화	온프레미스·폐쇄망 LLM	커리큘럼 13주
22	6. 전문·사업화	파인튜닝 vs RAG	커리큘럼 14주
23	6. 전문·사업화	금융권 AI 거버넌스	커리큘럼 15주

세션	파트	주제	출처
24	6. 전문·사업화	AX 사업화와 최종 발표	커리큘럼 16주

출처

- What is OpenClaw? (Operator)
- How OpenClaw Works (Medium)
- How to Use OpenAI Codex (Scrimba)
- Codex vs Claude Code in 2026 (Dupple)
- n8n Review 2026 (Beginners in AI)
- n8n MCP Server Complete Guide (MCP Directory)

도구 기능·가격은 빠르게 바뀌므로 각 단계 시작 시점에 공식 문서로 다시 확인하세요.

버전 기록

버전 규칙

현재 버전 v1.2 · 2026-11-12

기준 문서: AI 업무자동화·서비스·에이전트 개발 로드맵

- v1.x — 도구·문구·예제 보강, 출처 갱신처럼 세션 구성이 그대로인 변경
- v2.0 — 세션 구성이나 순서가 바뀌는 변경
- 로드맵 문서의 '버전 이력', 이 파일, CHANGELOG.md, git 태그를 같은 번호로 맞춥니다

2026-11-12

v1.2

- 24세션 모든 실습에 Python · Java 트랙 제공 (실습 탭에서 언어 전환). n8n · 코딩 에이전트 · Next.js 세션은 Python · Java 트랙을 새로 추가
- Java 실습: JDK 21 + Maven Wrapper, Spring Boot 4.1.2 · Spring AI 2.0.2 · Anthropic Java SDK 3.8.0 기준
- 실습 환경 구성 페이지 추가 (Windows · macOS 설치, API 키, 사내 폐쇄망 반입, 점검 스크립트, PDF · Markdown 가이드)
- 자료실 추가: 국내 · 해외 금융권(은행 · 증권 · 보험 · 캐피탈) · 공공 AI 도입 사례 26편
- 자료실 추가: 금융분야 AI 가이드라인 타임라인, 금융권 망분리 규제 타임라인 (출처 포함)

2026-10-29

v1.1

- 16주 AI 기술 스터디 커리큘럼과 6단계 로드맵을 병합해 6파트 24세션으로 재구성
- 세션마다 개념 · 이론(도식) · 실습 코드 · 실무 예제 카드 구성
- 스터디 사이트(ai-study)를 24세션 카드형으로 교체 (이전 16주 사이트는 claude/legacy-16week 브랜치로 보존)
- 이전 사이트 기능 이식: 카카오 로그인 · 운영자 승인 · 세션별 진도(개념·이론 / 실습 / 퀴즈) · 멤버 대시보드
- 세션마다 5번째 카드 '퀴즈·자료' 추가: 확인 퀴즈, 추천 영상·글, 실무 AI 용어집 연결
- 내 정보(이름·소속) 수정 기능, 시큐어 코딩 점검·조치 (SECURITY.md)
- 모바일 최적화: 하단 탭바, 한 줄로 넘기는 탭·필터·목차, 도식 가로 스크롤, 표 잘림 방지
- 카카오톡 등 링크 미리보기(Open Graph) 추가, 세션별 공유 링크 복사 버튼
- 상단 '버전 기록' 메뉴를 '로드맵'으로 바꾸고 로드맵 문서 전체(도식 9개)를 게시, 버전 기록은 그 아래로 이동

2026-10-29

v1.0

- 6단계 로드맵 초판: 업무자동화 → AI 코딩 → 풀스택 → MCP → 에이전트
- 회사별 툴체인 도구 지도 (Anthropic · OpenAI · Google · 오픈소스)
- Java/Spring 트랙 추가 (Spring AI 2.0, MCP Java SDK 1.0)